

Оглавление

1. Протокол HTTP. Структура запросов и ответов, методы запросов, коды ответов сервера, заголовки запросов и ответов.....	2
2. Язык разметки HTML. Особенности, основные теги и атрибуты тегов.....	5
3. Структура HTML-страницы. Объектная модель документа (DOM).....	7
4. HTML-формы. Задание метода HTTP-запроса. Правила размещения форм на страницах, виды полей ввода.....	10
5. Каскадные таблицы стилей (CSS). Структура - правила, селекторы. Виды селекторов, особенности их применения. Приоритеты правил. Преимущества CSS перед непосредственным заданием стилей через атрибуты тегов.	11
6. LESS, Sass, SCSS. Ключевые особенности, сравнительные характеристики. Совместимость с браузерами, трансляция в "обычный" CSS.....	13
7. Клиентские сценарии. Особенности, сферы применения. Язык JavaScript.	15
8. Версии ECMAScript, новые возможности ES6 и ES7.	16
9. Синхронная и асинхронная обработка HTTP-запросов. AJAX.....	18
10. Библиотека jQuery. Назначение, основные API. Использование для реализации AJAX и работы с DOM.....	20
11. Реализация AJAX с помощью SuperAgent.	22
12. Серверные сценарии. CGI - определение, назначение, ключевые особенности.	23
13. FastCGI - особенности технологии, преимущества и недостатки относительно CGI.	24
14. FastCGI сервер на языке Java.....	25
15. Что тоже нужно знать.....	27

1. Протокол HTTP. Структура запросов и ответов, методы запросов, коды ответов сервера, заголовки запросов и ответов.

Лекции, <https://habr.com/ru/articles/813395/> и гпт

HTTP – протокол для передачи гипертекста. Находится на прикладном уровне TCP/IP. Stateless протокол, состояние не сохраняется, для этого используются куки.

Куки – текстовые файлы, которые веб-сайт сохраняет на компьютер. Хранятся на жёстком диске компьютера в папке браузера. В них записана информация о посещении сайта. Они нужны, чтобы идентифицировать пользователя и заполнить какие-то данные специально для пользователя, например, языковые и региональные настройки. Вот как это работает. После того как вы вводите адрес нужной страницы в браузере, браузер ищет на устройстве файл cookie этого сайта. Если этот файл найден, то он отправляется на сервер ресурса. Сайт получает его и начинает использовать. Если браузер не находит cookie, сайт считает вас новым посетителем и просит разрешения создать файлы на устройстве.

Есть другой вариант хранения данных – **local storage** — это технология, которая позволяет веб-сайтам сохранять данные прямо в вашем браузере на неопределенный срок. В отличие от куки, эти данные не имеют срока действия и не отправляются на сервер с каждым HTTP-запросом. Данные будут лежать в браузере, пока их явно не уберут.

Для того, чтобы узнать к какой странице хочет обратиться пользователь, используется URL:



Так же есть URI и URN, их сравнения представлены в таблицы:

Название	За что отвечает
URL	Указатель местоположения ресурса, говорит, где и как найти ресурс <code>https://example.com/page?id=123</code>
URN	Уникальное имя ресурса, которое идентифицирует, но не говорит, где его найти <code>urn:isbn:0451450523</code>
URI	Универсальный идентификатор ресурса, может быть URL, URI, или чем-то, что их совмещает

Протокол передает данные по сети в открытом виде, без шифрования, с шифрованием передает https

Протокол HTTP работает в режиме запрос-ответ. Клиент, например, браузер, передает на сервер запрос к определенному ресурсу, например, Web-странице. Сервер в ответ отправляет клиенту этот ресурс или сообщение об ошибке, если ресурс передать нельзя:



Запрос HTTP состоит из 3 частей:

- Запрос
- Заголовки
- Тело запроса

Во второй строке указывается заголовок Host. Этот заголовок является обязательным в версии HTTP/1.1, в нем задается доменное имя сервера, к которому направлен запрос. Сейчас активно используется виртуальный хостинг: один Web-сервер на одном IP-адресе может обслуживать несколько сайтов с разными доменными именами. Чтобы понять, какому именно сайту предназначен запрос, нужно указать доменное имя сайта в заголовке Host.

Запрос состоит из:

- Метод HTTP
- Адрес ресурса, который хотят загрузить
- Версия протокола

Метод HTTP говорит о том, какое действие с ресурсом мы хотим совершить

Вот все методы HTTP:

Название метода	Описание метода
GET	Запрос на получение(чтение) ресурса с сервера
PUT	Заменяет весь ресурс новыми данными
HEAD	GET но без тела, только заголовки
OPTIONS	Спрашивает, какие методы и заголовки поддерживаются для ресурса или всего сервера
POST	Отправляет данные для создания или обработки ресурса
TRACE	Возвращает клиенту точно тот же запрос, который сервер получил, используется для отладки

DELETE	Удаляет указанный ресурс
--------	--------------------------

Ресурс – это любая сущность, которая находится по указанному URI после названия метода в заголовке HTTP

Наиболее частые заголовки:

От клиента:

Заголовок	Назначение
Host	Указывает домен и по необходимости порт сервера
User-Agent	Сообщает серверу, кто делает запрос: браузер, ОС, бот и тд
Accept	Говорит какие типы контента клиент готов принять
Connection	Управляет соединением, обычно keep-alive
Authorization	Передаёт данные авторизации
Content-Type	Указывает тип данных в теле запроса
Content-Length	Размер тело запроса в байтах

От сервера:

Server	Указывает тип сервера
Content-Type	Указывает тип данных в теле ответа
Content-Length	Размер тела ответа в байтах
Location	Указывает адрес перенаправления при коде 3xx
Set-Cookie	Отправляет клиенту новые куки

Ответ HTTP состоит из:

- Статус ответа
- Заголовки
- Тело сообщения

Статус ответа состоит из:

- Версия протокола
- Код статуса ответа

Есть обязательные 2 строки, которые идут после статуса:

- Content-Type: тип содержимого, которое передается в теле
- Content-Length: размер содержимого в байтах

Коды статусов:

- 1xx – информация, запрос получено, обработка продолжается
- 2xx – успешное выполнение
- 3xx – перенаправление, для выполнения запроса, надо выполнить дополнительные требования
- 4xx – ошибка клиента
- 5xx – ошибка сервера

2. Язык разметки HTML. Особенности, основные теги и атрибуты тегов.

Лекции, <https://doka.guide/html/> и гпт

HTML – стандартный язык разметки документов. Интерпретируется браузером и отображается в виде документа. *Браузер* — программа, отображающая HTML-документ в его отформатированном виде.

Особенности:

- Вложенность элементов
- Хз че тут еще написать

Основные теги:

Структурные:

- `<html>` - корневой элемент
- `<head>` - метаданные (стили, скрипты, заголовки)
- `<body>` - содержимое страницы
- `<header / footer>` - шапка / «подвал» страницы
- `<main>` - основное содержимое страницы
- `<h1 – h6>` - заголовки (по уровню важности)
- `<p>` - абзац

Ссылки и изображения:

- `<a href="...">` - ссылка
- `` - изображение, alt – надпись если изображение не загрузится

Списки и таблицы:

- `<ul>` - нумерованный список
- `<ol>` - нумерованный список
- `<li>` - элемент списка
- `<table>` - корневой элемент таблицы
- `<thead>` - заголовок таблицы
- `<tbody>` - содержимое таблицы
- `<tr>` - строка таблицы
- `<th>` - заголовок столбца
- `<td>` - ячейка таблицы

Ввод:

`<input type="text / password / radio / email">` - поле ввода

`<select>`- выпадающий список

`<option>` - определяет отдельный пункт в списке

Атрибуты тегов:

Почти для всех тегов:

- `id` – идентификатор
- `class` – класс для стилизации в `css` / `js`
- `title` – всплывающая подсказка

Ввод данных:

- `name` – имя элемента
- `value` – значение
- `placeholder` – текст-подсказка
- `required` – обязательное поле

Ссылки и медиа:

- `href` – url ссылка
- `src` – источник медиа
- `alt` – текст, если медиа не подгрузится

3. Структура HTML-страницы. Объектная модель документа (DOM).

Лекции

Структура HTML-страницы:

- Страница состоит из элементов
- В начале и в конце элемента есть теги, они могут быть пустыми
- Теги могут иметь атрибуты
- Элементы могут быть вложенными
- Документ должен начинаться со строки `<!DOCTYPE HTML...>`, хотя сейчас это вроде необязательно
- Начало и конец документа обозначаются тегами `<html>`, `</html>`, внутри этих тегов должны находиться хотя бы `<head> ... </head>` и `<body> ... </body>`

DOM – представление html документа в виде дерева узлов(объектов), которые имеют отношение родитель-потомок. Используется для динамического изменения страниц html например с помощью js

Основные операции с DOM

1. Поиск элементов

javascript

 Copy

 Download

```
// По ID
const element = document.getElementById('myId');

// По классу
const elements = document.getElementsByClassName('myClass');

// По тегу
const paragraphs = document.getElementsByTagName('p');

// Современные методы
const element = document.querySelector('#myId');
const elements = document.querySelectorAll('.myClass');
```

2. Изменение содержимого

javascript

 Copy

 Download

```
const title = document.getElementById('title');

// Изменить текст
title.textContent = 'Новый текст';

// Изменить HTML
title.innerHTML = '<strong>Жирный текст</strong>';

// Изменить атрибут
title.setAttribute('class', 'new-class');
```


3. Создание новых элементов

javascript

 Copy

 Download

```
// Создаем новый элемент
const newParagraph = document.createElement('p');
newParagraph.textContent = 'Я новый абзац!';

// Добавляем на страницу
document.body.appendChild(newParagraph);
```

4. Обработка событий

javascript

 Copy

 Download

```
const button = document.getElementById('myButton');

button.addEventListener('click', function() {
  alert('Кнопка нажата!');
});
```

4. HTML-формы. Задание метода HTTP-запроса. Правила размещения форм на страницах, виды полей ввода.

HTML-формы предназначены для обмена данными между пользователем и сервером

Метод HTTP задается атрибутом `method` внутри тега `<form>`

Атрибуты формы:

- `action` – указывает, куда отправлять данные
- `method` – указывает метод HTTP-запроса
- `enctype` – указывает тип кодирования данных
- `target` – определяет, где откроется результат

Правила размещения формы на странице:

- Границы формы задаются тегами `<form>...</form>`
- Документ может содержать сколько угодно форм, но одновременно может быть отправлена только одна из них
- Форма должна быть логически изолирована, каждая форма должна быть в отдельном теге `<form>`
- Вложенные формы запрещены

Виды полей ввода:

- `text` – однострочный текст
- `password` – скрывает вводимые символы
- `email` – проверяет формат email
- `number` – только числа
- `tel` – телефонный номер
- `url` – проверяет формат url
- `search` – поле поиска
- `checkbox` – множество вариантов
- `radio` – один вариант
- `range` – от минимума до максимума
- `button` – обычная кнопка
- `select` – выпадающий список

5. Каскадные таблицы стилей (CSS). Структура - правила, селекторы. Виды селекторов, особенности их применения. Приоритеты правил. Преимущества CSS перед непосредственным заданием стилей через атрибуты тегов.

CSS – технология для описания внешнего вида документа, написанного языком разметки. Используется для задания цветов, шрифтов и других аспектов представления документа.

Структура:

- Таблица стилей состоит из набора правил
- Каждое правило состоит из набора селекторов и блока определений

Виды селекторов:

Базовые:

- По тегу – выбирает все элементы указанного типа
`p {color: blue;}`
- По классу – выбирает все элементы с атрибутом class
`.class_name {font-size: 18px;}`
- По идентификатору – выбирает элемент с уникальным id
`#id_name {background: light-gray;}`
- Универсальный – выбирает все элементы на странице
`* {}`

Групповые – применяет одинаковые стили ко всем элементам

- `h1, h2, h3 {color: navy;}`

По отношениям:

- Потомок - `<p>` внутри `div` на любом уровне вложенности
`div p {color: green;}`
- Дочерние – только `<p>`, которые находятся прямо внутри `<div>`
`div > p {color: red;}`
- Соседние – применяется к `<p>` следующему сразу за `<h1>`
`h1 + p {margin-top: 0}`
- Все следующие элементы – применяется ко всем `<p>` после `<h1>` на том же уровне
`h1 ~ p {color: blue;}`

Псевдоклассы:

- При наведении – элемент под курсором мыши
`a:hover {color: red;}`
- При нажатии
`button:active {}`
- При фокусе – элемент в фокусе клавиатуры или мыши
`input:focus {}`

Приоритеты:

- Каскадные означает, что стили применяются по **принципу приоритета**, если один и тот же элемент, оформлен несколькими способами то браузер выбирает какой стиль применить по следующей иерархии:
 1. Свойство задано при помощи `!important`
 2. Стиль прописан напрямую в теге
 3. Наличие идентификатора (`#id`) в селекторе
 4. Количество классов и псевдоклассов (`:pseudoclass`) в селекторе
 5. Количество имен тегов в селекторе
- Имеет значение и относительный порядок, стиль добавленный позже имеет приоритет

Преимущества CSS:

- Разделение структуры и оформления
- Повторное использование стилей 1 css, можно подключить к множеству страниц
- В css, чтобы изменить цвет, достаточно поменять 1 строчку в html бы пришлось менять каждую строчку, где это нужно
- Поддержка псевдоклассов
- Браузер загружает .css файл 1 раз и кэширует его, поэтому стили будут быстрее подгружаться
- Поддержка иерархии стилей
- Единый стандарт, который поддерживают большинство браузеров

6. LESS, Sass, SCSS. Ключевые особенности, сравнительные характеристики.

Совместимость с браузерами, трансляция в "обычный" CSS.

CSS-препроцессор — это надстройка над обычным CSS, которая добавляет новые возможности (переменные, функции, вложенность). Препроцессорный код не понимается браузером напрямую. Он транслируется (компилируется) в обычный CSS.

LESS (Leaner CSS) — препроцессор, синтаксис которого похож на обычный CSS. Написан на JavaScript, может работать прямо в браузере или через Node.js.

Особенности:

- Синтаксис близкий к CSS
- Переменные с символом @
- Поддерживает вложенность, функции, операции
- Компиляция через Node.js или автоматически при сборке

Sass (Syntactically Awesome Style Sheets) — один из первых и самых мощных препроцессоров.

Существует два синтаксиса:

- Sass – старый
- SCSS – новый, полностью совместим с CSS синтаксисом

♦ 3. Сравнение LESS, Sass и SCSS					
Возможность / Признак	LESS	Sass	SCSS		
Расширение файла	.less	.sass	.scss		
Основа языка	JavaScript	Ruby	Sass (новый синтаксис)		
Совместимость с CSS	Почти полная	Нет (особый синтаксис)	Полная		
Символ переменной	@	\$	\$		
Вложенность	✓	✓	✓		
Миксины	✓	✓	✓		
Условия / циклы	⚠ (ограниченно)	✓	✓		
Импорт модулей	✓ (@import)	✓ (@use , @forward)	✓ (@use , @forward)		
Поддержка функций	✓	✓ (много встроенных)	✓		
Поддержка браузерами	Через компиляцию	Через компиляцию	Через компиляцию		
Простота освоения	★★★★★	★★★	★★★★★		
Производительность	Быстрее (JS)	Немного медленнее (Ruby)	Современный стандарт		

Не один из этих препроцессоров не поддерживается браузерами напрямую, нужно скомпилировать в .css.

◆ 5. Трансляция (компиляция) в обычный CSS		
Инструмент	Для чего	Пример команды
lessc	Компилятор LESS (Node.js)	<code>lessc styles.less styles.css</code>
sass	Компилятор Sass/SCSS	<code>sass style.scss style.css</code>

7. Клиентские сценарии. Особенности, сферы применения. Язык JavaScript.

Клиентские сценарии – программы, которые выполняются на стороне клиента, в браузере пользователя, а не на сервере.

Особенности:

- Выполняется в браузере
- Язык почти всегда JS иногда TypeScript, который компилируется в JS
- Многие операции (валидация, анимация, взаимодействие с интерактивными модулями) не требует обращения к серверу
- Поведение программы зависит от конкретного браузера
- Не имеет прямого доступа к файлам или ОС пользователя

Сферы применения:

- Интерактивность интерфейса – всплывающие окна, вкладки, анимации
- Валидация
- Работа с DOM
- Хранение данных в LocalStorage

JavaScript - объектно-ориентированный скриптовый язык программирования. Используется для придания интерактивности веб-страницам.

Основные особенности архитектуры:

- Динамическая типизация – тип переменной не определяется заранее
- Слабая типизация – автоматически преобразует типы, когда это нужно для выполнения
- Автоматическое управление памятью
- Прототипное программирование – нет классов как в java, все основано на объектах и прототипах. Наследование происходит через цепочку прототипов
- Функции – полноценные объекты

Особенности синтаксиса:

- Все имена регистрозависимы
- Комментарии как в java
- Инструкции разделяются ;, но при переносе строки необязательно
- Переменные объявляются ключевым словом let или const
- {} обозначают блок кода
- Условные операторы как в java

8. Версии ECMAScript, новые возможности ES6 и ES7.

ESMA – стандарт языка, который используется в ядре JS

Особенности:

- 5 примитивных типов: Numeric, Boolean, String, Null, Undefined
- Объектный тип данных – Object
- 15 различных видов инструкций
- Блок не ограничивает область видимости переменной
- Если переменная объявляется не в функции, то она является глобальной
- Функция – объект

```
// объявление функции
function sum(arg1, arg2) {
    return arg1 + arg2;
}

// задание функции с помощью инструкции
var sum2 = function(arg1, arg2) {
    return arg1 + arg2;
};

// задание функции с использованием
// объектной формы записи
var sum3 = new Function("arg1", "arg2",
    "return arg1 + arg2;");
```

ES6 добавляет много новых возможностей:

- Два новых ключевых слова let и const:
 - let позволяет объявить переменную видимость, которой является блок
 - const объявляет константу
- Template Literals:
const first = 'Adrian';
const last = 'Mejia';
console.log(`Your name is \${first} \${last}.`);
- Деструктуризация – особый синтаксис присваивания, при котором можно присвоить массив или объект сразу нескольким переменным, разбив его части
const array = [1, 2, 3, 4];
const [first, ,third] = array;
console.log(first, third); // 1 3
- Так же можно делать обмен значениями с помощью деструктуризации
let a = 1;


```
let b = 2;  
[a, b] = [b, a];  
console.log(a, b); // 2 1
```

- Новый синтаксис для описания и инициализации объекта

```
class Animal {  
  constructor(name) {  
    this.name = name;  
  }  
  speak() {  
    console.log(this.name  
      + ' makes a noise.');  }  
}  
  
const animal =  
  new Animal('animal');  
// animal makes a noise.  
animal.speak();
```

- Новые ключевые слова `extends` и `super`
- Появление промисов, раньше были колбэки. Как это работает, код в JS выполняется синхронно и последовательно, но если нужно подождать что-то, например ответ от сервера. Появляются асинхронные операции промисы и колбэки.

Callback – функция, которая передается другой функции как аргумент и выполняется позже, когда операция завершится. Если нужно будет несколько асинхронных операций, то будет вложенность в колбэках, и код будет нечитабельным. Придумали промисы

Promise – объект, который обещает вернуть результат в будущем. Есть 3 состояния (ожидание, успешное выполнение, ошибка). Они позволяют выполнять асинхронные операции друг за другом, без вложенности.

Promise

```
.then() сколько угодно можно писать .then()  
.catch();
```

- Появились стрелочные функции
`const sum = (a, b) => a + b;`
- Цикл с итератором
`const array = ['a', 'b', 'c', 'd'];
for (const element of array) { console.log(element);}`
- Параметры по умолчанию
`function point(x = 0, y = -1, isFlag = true)`

9. Синхронная и асинхронная обработка HTTP-запросов. AJAX.

При **синхронной обработке** запроса браузер ждет ответа от пользователя, прежде чем продолжить работу, блокирующий режим. При **асинхронной** запрос отправляется серверу, но работу сайта продолжается, когда придет ответ, сайт отобразит его для пользователя.

DHTML – Dynamic HTML способ создания интерактивного веб-сайта, используя сочетания:

- Статичный язык разметки HTML
- Выполняемого скриптового языка JS на стороне клиента
- CSS
- DOM

AJAX – подход к построению интерактивных пользовательских интерфейсов веб-приложений. Основан на фоновой обмене данными браузера с веб сервисом. При обмене данными веб-страница не перезагружается полностью

Основные принципы:

- Использование технологии динамического обращения к серверу без перегрузки всей страницы полностью, например:
 - XMLHttpRequest
 - Через динамическое создание тега <script>
- Использование DHTML для динамического изменения содержания страницы

Преимущества:

- Экономия трафика
- Уменьшение нагрузки на сервер
- Ускорение реакции интерфейса

Недостатки:

- Требуется JS встроенный в браузер
- Усложнение проекта

XMLHttpRequest набор API, позволяющие осуществлять HTTP-запросы к серверу без необходимости перезагружать страницу:

Данные можно пересылать в виде JSON, XML, HTML или просто неструктурированного текста. Обязательно текстовый формат, так как данные передаются по HTTP

WebSocket – протокол двухсторонней постоянной связи между браузером и сервером, не закрывается и позволяет общаться между собой в режиме реального времени.

Также еще одним плюсом является то, что при сервер может обращаться к браузеру без запроса.

Обмен данными ведется через отдельное TCP соединение

Поддерживается всеми современными браузерами

10. Библиотека jQuery. Назначение, основные API. Использование для реализации AJAX и работы с DOM.

jQuery – библиотека JS для разработки DHTML и AJAX приложений

- Упрощает доступ к элементам DOM
- Упрощает и унифицирует реализацию AJAX для разных браузеров
- Упрощает добавление визуальных эффектов
- Ключевым элементом API является функция \$ и ее синоним jQuery

9. Основные API jQuery (группы функций)

Категория	Примеры методов
DOM	<code>.html()</code> , <code>.text()</code> , <code>.append()</code> , <code>.remove()</code>
CSS	<code>.css()</code> , <code>.addClass()</code> , <code>.removeClass()</code> , <code>.toggleClass()</code>
События	<code>.on()</code> , <code>.click()</code> , <code>.hover()</code> , <code>.keyup()</code>
AJAX	<code>\$.get()</code> , <code>\$.post()</code> , <code>\$.ajax()</code>

Без jQuery (и без кроссбраузерности):

```
req = new XMLHttpRequest();
req.open("POST", "some.php", true);
req.onreadystatechange = processReqChange;
req.send(null);
if (req.readyState == 4) {
    if (req.status == 200) {
        alert( "Data Saved " );
    }
}
```

C jQuery:

```
$.ajax({
    type: "POST",
    url: "some.php",
    data: {name: 'John', location: 'Boston'},
    success: function(msg){
        alert( "Data Saved: " + msg );
    }
});
```

Взаимодействие с DOM

```
$("div.test")
    .add("p.quote")
    .addClass("blue")
    .slideDown("slow");
```

```
$("a").click(function() {
    alert("Hello world!");
});
```

11. Реализация AJAX с помощью SuperAgent.

SuperAgent – библиотека для выполнения AJAX запросов в JS.

request

.method_name(url) – отправка с помощью какого-то метода

.send(json) – отправка данных (тело запроса)

.set(заголовок, значение) – выставляет значение заголовков

.then() – можно вызывать так как возвращается Promise

.catch() – аналогично

.end() – отправляем запрос и ждем ответа, пока не будет вызван end, запрос не будет отправлен

Используется либо end, либо then / catch

12. Серверные сценарии. CGI - определение, назначение, ключевые особенности.

Серверные сценарии – программа, которая выполняется на стороне сервера:

- Обработывает запрос от браузера
- Работа с различными ресурсами
- Формирование и отправка ответа браузеру

CGI – простейший механизм вызова пользователем программ на стороне сервера

Особенности:

- Данные отправляются программе через HTTP запрос, сформированный браузером
- То какая программа будет вызвана обычно указано в URL
- Каждый запрос обрабатывается отдельным процессом программы
- Взаимодействие программы с браузером осуществляется с помощью stdin и stdout

Достоинства:

- Программа может быть написана на различных языках
- Исключены конфликты при параллельных процессах
- Падение CGI-сценария не приводит к падению всего сервера

Недостатки:

- Высокие расходы на создание нового процесса
- Плохая масштабируемость
- Может быть платформо-зависимым

13. FastCGI - особенности технологии, преимущества и недостатки относительно CGI.

FastCGI – развитие технологии CGI

- Все запросы могут выполняться одним процессом CGI-программы
- Веб сервер взаимодействует с процессом через TCP/IP, а не потоки ввода-вывода
- Для общения по TCP/IP используется бинарный протокол, обеспечивающий передачу любых текстовых данных
- Использует постоянное соединение между браузером и сервером

FastCGI > CGI:

- Быстрее – процессы не создаются заново при каждом запросе
- Экономия ресурсов
- Масштабируемость – нужно настроить несколько рабочих процессов для обработки большого потока запросов

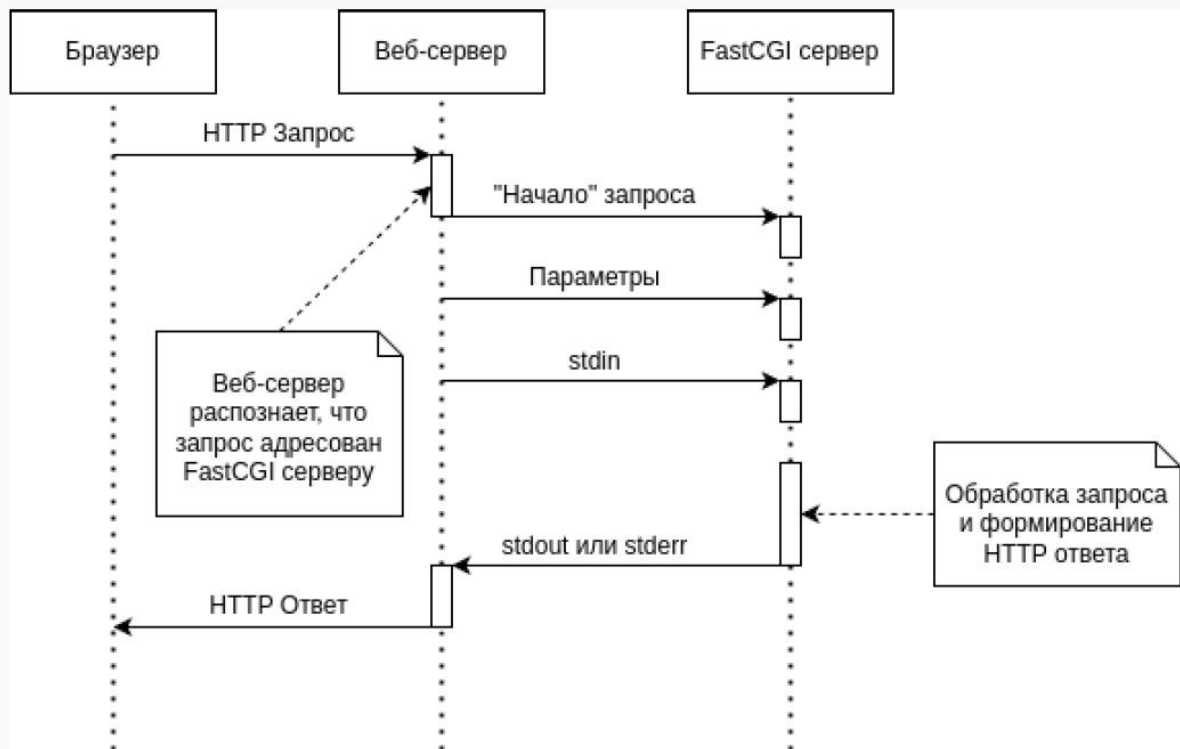
FastCGI < CGI:

- Сложнее в настройке – нужно управлять пулами процессов
- Зависшие процессы могут не завершаться автоматически

14. FastCGI сервер на языке Java.

FastCGI поддерживает набор запущенных процессов, либо отдельный FastCGI сервер.

Типы FastCGI пакетов



В лабе веб-сервер – Apache

Описание картинки:

1. Браузер передает HTTP-запрос на веб-сервер
2. Веб-сервер распознает, что запрос адресован FastCGI серверу
3. Открывается соединение к FastCGI серверу, возможно выбирается уже из существующего пула
4. Сообщаем FastCGI серверу, что начинается новый запрос
5. Передаем параметры (заголовки), если передается пустой пакет, значит передача закончена
6. Дальше передаем тело запроса с помощью STDIN, заканчиваем также пустой строкой
7. Обработка запроса и формирование ответа
8. Отправка ответа, если все прошло успешно, то через STDOUT, иначе – STDERR

9. Веб-сервер получает ответ от FastCGI и формирует полноценный HTTP-ответ для браузера
10. Дальше отправляет HTTP-ответ в браузер

FastCGI на Java:

```
FCGIInterface fcgiInterface = new FCGIInterface();
```

FCGIInterface – класс из библиотеки, который реализует магию перенаправление данных из стандартных потоков в потоки TCP сокетa.

FCGIInterface.request.params - объект Properties, через, который можно узнать список параметров, переданных веб-сервером

Наиболее значимые параметры:

- REQUEST_METHOD — GET, POST и т. д.;
- QUERY_STRING — параметры запроса после «?»;
- CONTENT_TYPE — значение заголовка запроса;
- CONTENT_LENGTH — значение заголовка запроса.

Для того, чтобы запустить сервер нужно также нужно настроить Apache конфиг, что это вообще такое? Это веб-сервер, глобально он просто передает запросы в java программа, которая уже их обрабатывает, дальше получаем ответ от сервера и отправляем его в браузер

15. Что тоже нужно знать

REST (Representational State Transfer) — это архитектурный стиль или набор принципов для построения веб-служб (API). Проще говоря — это способ договориться о том, как клиенты (например, ваше приложение) и серверы должны общаться друг с другом.

1. **Единообразие интерфейса:**

- **Ресурсы:** Вся сущность (пользователь, заказ, товар) рассматривается как ресурс, доступный по уникальному URL (например, `/api/users/123`).
- **Самодостаточные сообщения:** Каждый запрос от клиента содержит всю информацию, необходимую серверу для его обработки.
- **HTTP-методы:** Для взаимодействия с ресурсами используются стандартные HTTP-глаголы

2. **Отсутствие состояния (Stateless):** Сервер не хранит состояние клиента между запросами. Каждый запрос должен содержать всю необходимую информацию для его выполнения.

3. **Кэшируемость (Cacheable):** Ответы сервера должны явно указывать, можно ли их кэшировать, чтобы уменьшить количество запросов и повысить производительность.

4. **Клиент-серверная архитектура:** Четкое разделение обязанностей: клиент отвечает за пользовательский интерфейс, сервер — за хранение данных и бизнес-логику.

Если API построено в соответствии с принципами REST, его называют **RESTful API**.

RPS (Requests Per Second) — это метрика производительности, которая переводится как "запросов в секунду". Она показывает, сколько запросов данная система (сервер, база данных, API) может обработать за одну секунду. Это ключевой показатель для оценки нагрузки и производительности.